

TOI 21th at KMITL

Report from Scientific Committee

กรรมการวิชาการ

- เกรียงศักดิ์ ตรีประพิณ
- เจษฎา ตัณฑนุช
- ณัฐพงศ์ ศรีวัฒนศักดิ์
- ทัดดาว ปานสมบัติ
- ธนาวินท์ รักธรรมานนท์
- นัทที นิภานันท์
- บุญหทัย เครือแก้ว
- ปัญญนัท อ้นพงษ์
- มัตตัญญู ตั้งแจ้งก๊ก
- เลขาขวัญ งามประสิทธิ์
- วัชรพัฐ เมตตานันท
- วาโย ปุยะติ
- ศุภกร กิจวัฒนาชัย
- สิริวิชญ์ พงศ์นรินทร์
- สิริโชค จิงถาวรณ

Contest Report

Incident Day 1

- 08:30 เริ่มทำการสอบ
- 10:07 ตรวจพบว่า PDF ข้อ Tech Sprites มีการระบุค่า Time limit ผิด (ต้องเป็น 2.5 แต่ใน PDF เขียนไว้ว่าเป็น 2) ไม่มีผลกระทบต่อคะแนนใด ๆ
- 10:19 ตรวจพบว่า PDF ข้อ Dining Car มีการระบุค่าขอบเขตของข้อมูลผิด (ต้องเป็น $1 \leq N \leq 10^9$ แต่ต้องเป็น $2 \leq N \leq 10^9$) ไม่มีผลกระทบต่อคะแนนใด ๆ
- 11:16 มีคำถามเกี่ยวกับผลการตรวจข้อ Dining Car เนื่องจากหากใช้ 90 ครั้งพอดี สูตรจะคำนวณคะแนนออกมาเป็น 0 พอดี ทำให้ผลการตรวจถือเป็น “Not correct” แต่พอผู้เข้าแข่งขันกดเข้าไปดูใน detail จะเห็นว่า “Output is correct” เนื่องจากโปรแกรมทำงานถูกต้องทุกอย่างแต่ได้คะแนน 0 ได้เขียนชี้แจงในประกาศแล้ว
- 12:06 ผู้เข้าแข่งขันแจ้งว่า subtask แรก ๆ ของข้อ Dining มีการตรวจแบบ partial (ซึ่งไม่ตรงกับที่โจทย์กำหนด) แก้ไขเสร็จเวลา 12:36 ทำให้มีผู้เข้าแข่งขัน 8 คนได้คะแนนเพิ่มเติม และระหว่างการแก้ไขทำให้ระบบไม่สามารถ feedback ผลการตรวจได้ จึงต่อเวลาให้ผู้เข้าแข่งขันทุกคน 15 นาที

Incident Day 2

- 08:35 เริ่มสอบ
- 09:25 ตรวจพบคำผิดใน PDF ข้อ Crystal เรื่อง Time limit ในระบบเป็น 2 แต่ใน PDF เป็น 1
- 08:57 ตรวจพบคำผิดใน PDF ข้อ Duty Free เรื่องหมายเลขช่องในรูปไม่ตรงกับคำอธิบาย
- 09:25 แก้ไข PDF เรียบร้อย ทำประกาศแจ้งนักเรียน
- 09:58 ตัวอย่างข้อ Crystal มีเลขพลังค์ซ้ำ ปรับแก้ไฟล์ PDF

PROBLEM REPORT

Solution Count

มีกรรมการที่
คนช่วยทำ

เฉลยที่ได้คะแนน
บางส่วนมีอีกอัน

โอกาสได้คะแนน
ย่อย ๆ

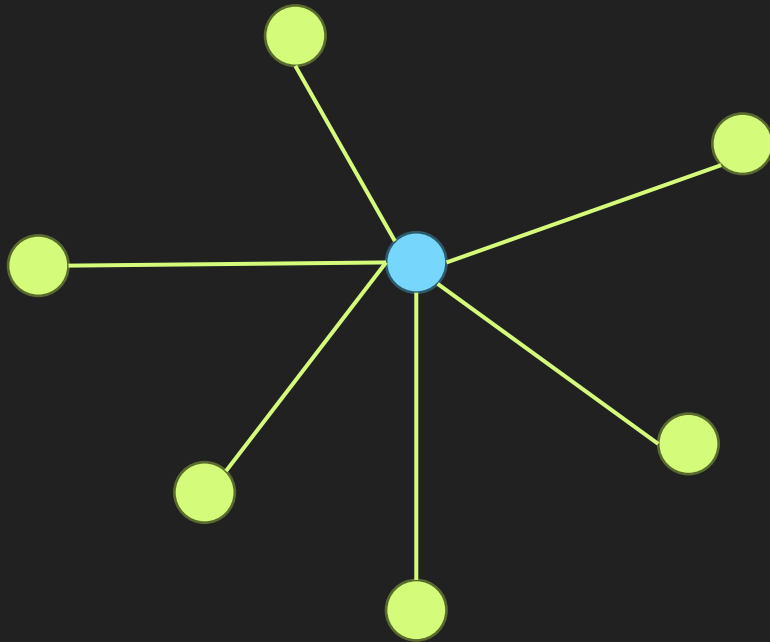
Problem	Official Solution	Subtask solution	#Subtask	# Testcases
D1: Tech Sprite	4	4	6	45
D1: Quartet	4	10	7	47
D1: Dining Car (interactive)	3	10	6 (variable scoring)	96
D2: Crystal	3	6	7	80
D2: Duty Free (function)	3	9	7	97
D2: Kang (function)	2	11	7	90

Tech Sprites Solution

- ให้สังเกตว่าในกลุ่มบ้านต่าง ๆ Tech Sprites จะออกมาแบบเรียงกันอยู่แล้ว
- ปัญหาอยู่ที่ Tech Sprites ที่อยู่คนละกลุ่มบ้าน
- เราต้องสร้างทางเชื่อมให้บ้านทั้งหมดกลายเป็นกลุ่มเดียวกัน
- สร้างกราฟ ให้ปมคือบ้านน้อย เส้นเชื่อมคือ รางเชื่อมตรง
- หาจำนวน Connected Component แล้วสร้างทางเชื่อม Connected Component ต่าง ๆ เข้าด้วยกัน
 - นับจำนวน Connected Component (สมมติให้เป็น P) ให้ตอบ $P-1$

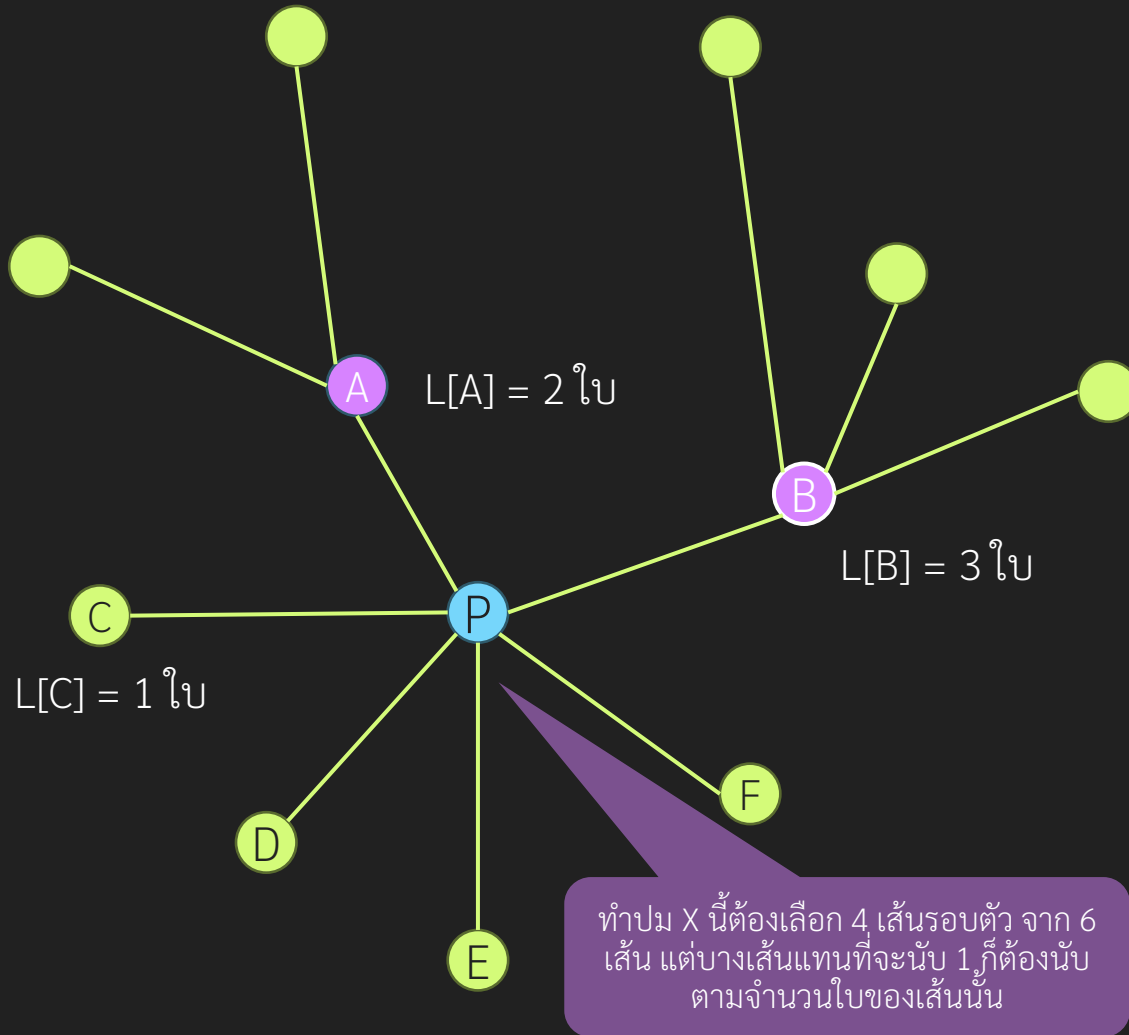
Quartet Solution

- แต่ละปมที่เป็น internal node ต้องตอบว่าเราสามารถเลือก “ใบ” มาจาก 4 ที่ครอบตัวได้กี่แบบ
- กรณีพื้นฐานคือรูป star



คำตอบคือ 6 เลือก 4

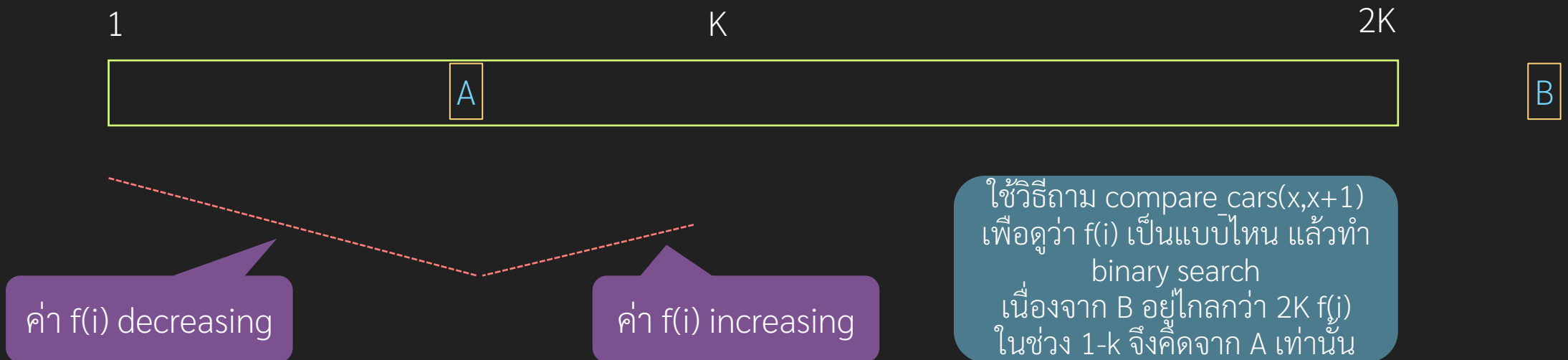
Quartet Solution



- กรณีทั่วไป คิดจากแต่ละ internal node (ศูนย์ตำบล) ให้สังเกตว่า แต่ละศูนย์ตำบลนั้นจะมี “กลุ่มอพยพ” ที่ไม่ซ้ำกันเลย ดังนั้นคำตอบคือผลรวมของ “กลุ่มอพยพ” ที่ใช้ internal node นั้นเป็นศูนย์ตำบล
- สำหรับแต่ละ internal node ให้ถามว่า แต่ละ edge ที่เชื่อมกับ internal node นั้นมีใบอยู่กี่ใบ (ให้ $L[x]$ คือจำนวนใบของ subtree x ที่เชื่อมกับ internal node P นั้น)
 - ให้ Q คือจำนวนเส้นเชื่อมที่ต่อกับปม P
 - คำนวณ $L[]$ โดยใช้ DFS นับ ๆ เอาใน $O(N)$
 - คำตอบจำนวน “กลุ่มอพยพ” ของ internal node นั้นคือ ผลรวมของ $L[i] * L[j] * L[k] * L[m]$ (โดยที่ $0 \leq i < l < k < m < Q$)
- ถ้าคำนวณ $L[i] * L[j] * L[k] * L[m]$ ตรง ๆ ใช้ ใช้ DFS เพื่อนับจำนวนใบที่อยู่ใต้ปมนั้น ๆ ใช้ $O((\text{จำนวน edge ของ } x)^4)$
 - ทำให้ดีขึ้นได้โดยใช้ Dynamic Programming
 - ให้ $DP[a][b]$ คือ ผลรวมของ ผลคูณของการเลือก a ตัวจาก Q ตัว โดยใช้เฉพาะตัวที่ index $\geq b$
 - $DP[1][x] = L[x] * DP[1][x+1]$
 - $DP[i][x] = L[x] * DP[i-1][x+1]$
 - คำตอบคือ $DP[4][0] * DP[4][1] * \dots * DP[4][Q-4]$
 - ใช้ $O(\text{เส้นเชื่อม})$

Dining Car Solution

- ให้ $f(i)$ คือ ระยะห่างจากตู้ i ไปยังตู้อาหารใกล้สุด
 - เราเรียกใช้ $f(i)$ ไม่ได้ แต่ เรียก $\text{compare_cars}(P,Q)$ ซึ่งเปรียบเทียบ $f(P)$ กับ $f(Q)$ ให้
- ลองพิจารณากรณีง่ายก่อน
 - สมมติว่าตู้ A อยู่ในช่วงตู้ 1 ถึง K แน่ ๆ และตู้ B อยู่ไกลกว่า $2K$



Dining Car Solution

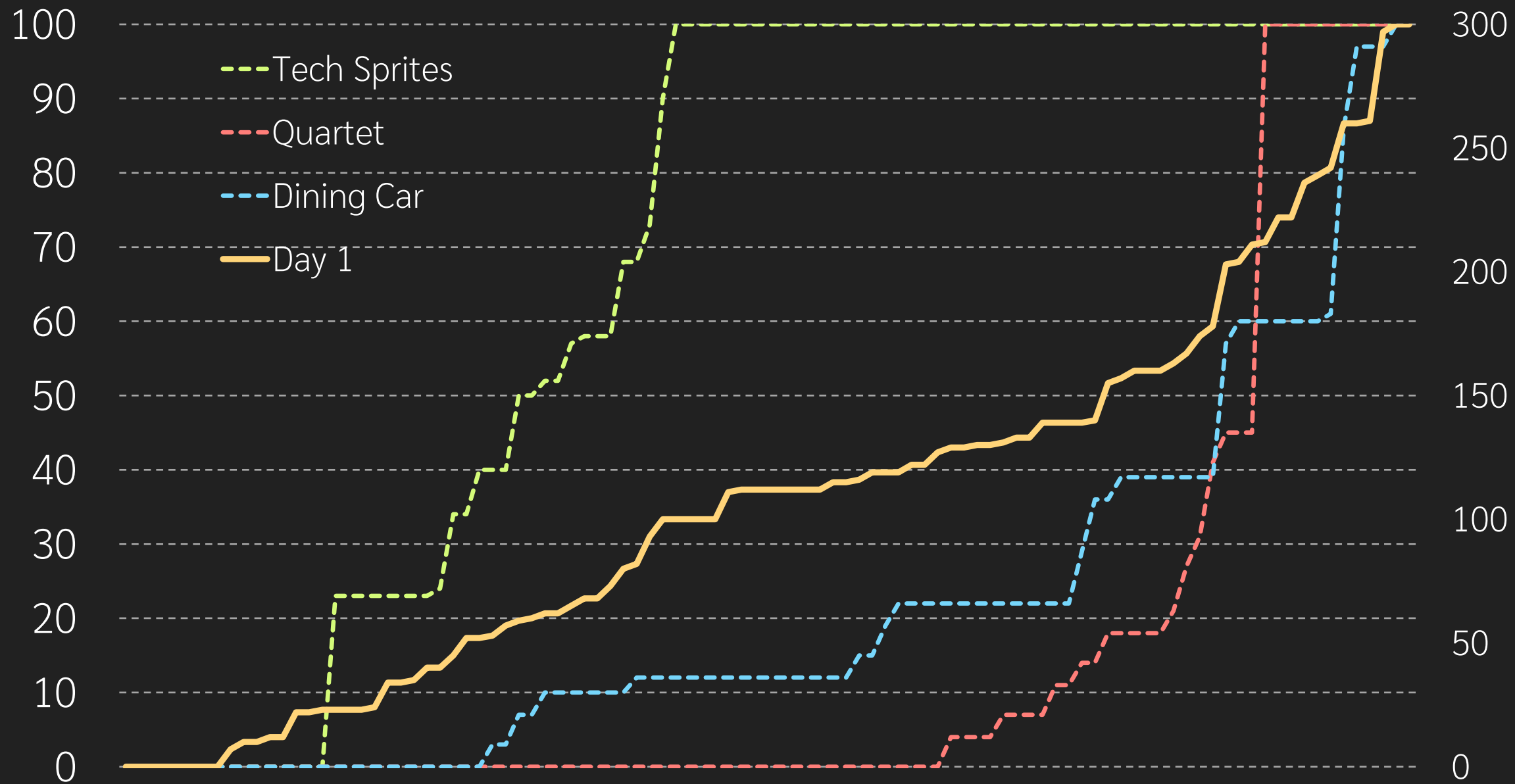
- ให้ $[L,R]$ คือช่วงที่ตู้อาหารสามารถอยู่ได้ เราจะปรับ L,R ให้ลดลงเรื่อย ๆ แบบ binary search
- เริ่มต้นให้ $L = 1$ และ $R = N$, $M = (L+R)/2$
- ถาม $\text{compare_cars}(L,R)$

ผล $\text{compare_car}(L,R)$	ความหมาย
0 แปลว่า $f(L) = f(R)$	A อยู่ในช่วง (L,M) และ B อยู่ที่ $2M-A$ ใช้ Binary Search หา A และ B ได้
-1 แปลว่า $f(L) < f(R)$	• ให้ถาม $\text{compare_cars}(M,M+1)$ ต่อ • ถ้าได้ -1 แสดงว่าทั้ง A, B อยู่ในช่วง $[L,M]$ • ถ้าได้ +1 แสดงว่า B อยู่ใน $[M+1,R]$, A อยู่ $[L, 2M-B]$ • B ก็หาด้วย Binary Search • สำหรับ i ในช่วง $[L, 2M-B]$ (!!!) • $f(i) = f(i+1)$ ถ้า $A = 2i+1-B$ • $f(i) < f(i+1)$ ถ้า A อยู่ในช่วง $(2i+2-B, i)$ ซึ่งสามารถทำ binary search หาค่า A ได้

Dining Car Solution

ผล <code>compare_car(L<R)</code>	ความหมาย
1 แปลว่า $f(L) > f(R)$	• ให้ถาม <code>compare_cars(M,M+1)</code> ต่อ • ถ้าได้ 0 แสดงว่า A อยู่ใน $[L,M]$ และ $B = 2M+1-A$ • ใช้ Binary Search หา • ถ้าได้ -1 แสดงว่า A อยู่ใน $[L,M]$ และ B อยู่ในช่วง $[2M+2-A,R]$ • A ใช้ Binary Search หา • B ทำแบบเดียวกับ (!!!) • ถ้าได้ +1 แสดงว่า แสดงว่าทั้ง A, B อยู่ในช่วง $(M+1, R)$

- at worst ต้องเรียก `compare_cars` สองครั้งถึงจะลดช่วง L,R ได้ครึ่งหนึ่ง
- bound คือ $2 \log_2(n)$



Score Summary – Day 1

โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Tech Sprites	57 00:14 min	83	6,830	817
Quartet	12 00:26 min	36	1,639	594
Dining Car	2 01:43 min	71	2,179	609

เวลาจากการเริ่มแข่งขัน
ที่มีคนได้เต็มคนแรก

มี 8 คนไม่ได้คะแนนเลย

กำลังพิจารณาตัวนี้

Crystal Key Idea

- Longest Increasing Subsequence (LIS) หลายชุดที่เชื่อมต่อกันได้
- ถ้าไม่ข้ามจักรวาลเลย คำตอบก็คือ LIS ของผลึกในจักรวาลนั้น ปัญหาคือจะคำนวณกรณีที่ข้ามจักรวาลอย่างไร
- ลองพิจารณา LIS แบบมาตรฐานก่อน วิธีทำปรกติคือ เราจะมี data structure ที่เก็บคู่ (a,b) เพื่อบอกว่ามี LIS ที่ตัวสุดท้ายมีค่าเป็น a และมีความยาวเป็น b อยู่ เราจะหา LIS โดยพิจารณาทีละตัวแล้วปรับปรุง data structure นี้ไปเรื่อย ๆ

10	50	30	60	40	20	50	30	70
----	----	----	----	----	----	----	----	----

ตัวสุดท้าย	ความยาว LIS
10	1
30	2
60	3



ตัวสุดท้าย	ความยาว LIS
10	1
30	2
40	3

LIS ปรกติจะใช้ data structure ที่หา, ลบ, แทรกเร็ว ๆ เช่น set

Crystal Solution

- ในข้อนี้ เราจะคำนวณ $L[w]$ ซึ่งเป็น data structure สำหรับ LIS ที่ผลึกก้อนสุดท้ายอยู่ ณ จักรวาล w
- พิจารณาผลึกตามลำดับเวลา t จากน้อยไปมาก ให้ผลึกมีพลังเป็น p อยู่ในจักรวาล w
 - เราจะปรับปรุง $L[w]$ ไปคล้าย ๆ กับ LIS มาตรฐาน แต่จะลอง “กระโดด” มาจากจักรวาลอื่นด้วย
 - ให้ G เป็น Data Structure อีกตัวที่เก็บ LIS ที่จบที่จักรวาลใดก็ได้
 - คำนวณ $L[w]$ ใหม่โดยเลือกดีสุดจาก
 - ใช้คู่ (a,b) ใน $L[W]$ ที่ b มากสุดที่ $a < p$ (เหมือนทำ LIS ปกติบนจักรวาล W)
 - ใช้คู่ (a,b) ใน G ที่ b มากสุดที่ $a < p$ แต่ LIS ที่ได้จากอันนี้จะมีความยาวลดลงไป K (cost ในการกระโดด)
 - ให้ (a',b') คือ LIS ดีสุดจากสองข้อข้างต้น
 - คำนวณ G ใหม่โดยใช้ (a',b') ไปปรับปรุง

Crystal Subtask

- Subtask 1 $O(2^N)$

- ลองทุก combination ของก้อนผลึก

- Subtask 2

- มีจักรวาลเดียว เท่ากับทำ LIS เฉย ๆ

- Subtask 3

- มีหลายจักรวาลแต่ไม่เสียค่ากระโดด ดังนั้นเสมือนกับทุกผลึกอยู่จักรวาลเดียวกัน

- Subtask 4, 5, 6

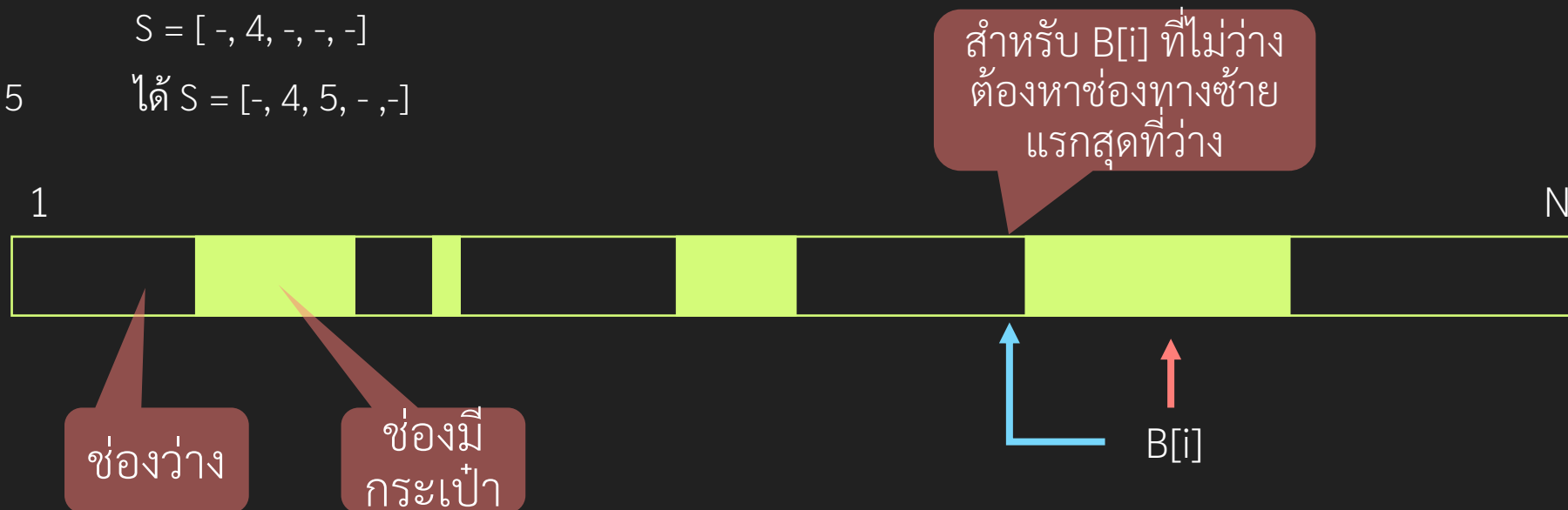
- มองเป็น directed acyclic graph เชื่อม crystal เวลา t กับเวลา $> t$ ที่มีเส้นเชื่อมเฉพาะกรณีที่ค่าพลังมากขึ้น
แล้วหา longest path

Duty Free Key Idea

- ให้ $S[i]$ คือ สถานะของ “ชั้นวางกระเป๋า” $S[i] = x$ หมายความว่ากระเป๋าของลูกค้าคนที่ x ถูกวางที่ช่องหมายเลข i เช่น
 - $S = [1, 4, 2, 3]$ แปลว่า ตำแหน่งแรกเราวางกระเป๋าของลูกค้าคนที่ 1 ตำแหน่งสองเราวางกระเป๋าของลูกค้าคนที่ 4,
 - $S = [-, 1, 3, -]$ แปลว่า ช่องแรกและช่องสุดท้ายไม่มีกระเป๋าว่าง ส่วนช่องสองเป็นกระเป๋าของลูกค้าคนที่ 1
- Observation
 - สำหรับลูกค้าคนที่ i เราอยากจะวางกระเป๋าของเขาให้ “ไกล” ที่สุดที่ไม่เกิน $B[i]$ (เพื่อคนที่ใกล้ ๆ ไว้ให้คนที่ไม่อยากจะเดินไกล)
 - สมมติ $B = [4, 5, 4, \dots]$ เราอยากวางกระเป๋าเป็น $[-, -, 3, 1, 2]$
 - Solution: เป็น Greedy ก็คือวางด้วยวิธีดังกล่าวสำหรับลูกค้าตั้งแต่คนที่ 1 ไปถึงคนที่ N ถ้าวางได้ ก็แปลว่าไม่ต้องจัดเรียงกระเป๋าใด ๆ เลย
 - ถ้าเราไม่สามารถวางให้สำหรับลูกค้าคนที่ i ได้ เช่น $B = [3, 1, 2, 1, 1]$ จะเห็นว่าเราวางเริ่มต้นเป็น $S = [2, 1, 3, 4, 5]$ ก็จะขายได้จนถึงคนที่ 4 ก็ต้องจัดเรียงใหม่เฉพาะกระเป๋าของตั้งแต่คนที่ 4 เป็นต้นไป

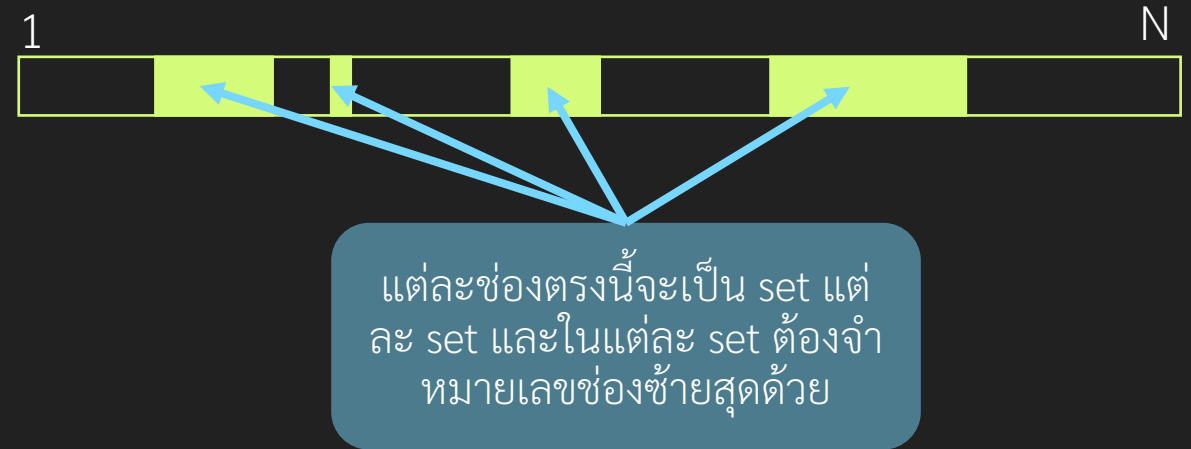
Duty Free Key Idea

- โจทย์จริง ๆ คือการ simulate การวางกระเป๋าว່ว่าเราสามารถหาตำแหน่ง **ใกล้สุดที่ไม่เกิน $B[i]$** ได้จากสถานะของ S ปัจจุบันหรือเปล่า (และ reset state หลังจากไม่สามารถวางกระเป๋าของคนที่ i ได้)
- สมมติให้ $B = [1, 3, 3, 2, 3]$
- ณ $i = 1$ ได้ $S = [1, -, -, -, -]$
- ณ $i = 2$ ได้ $S = [1, -, 2, -, -]$
- ณ $i = 3$ ได้ $S = [1, 3, 2, -, -]$
- ณ $i = 4$ จะวางไม่ได้แล้ว ต้องรอคนที่ 1 – 3 มาซื้อเสร็จ แล้วค่อย เรียงใหม่ ได้เป็น $S = [-, 4, -, -, -]$
- ณ $i = 5$ ได้ $S = [-, 4, 5, -, -]$



Duty Free Official Solution

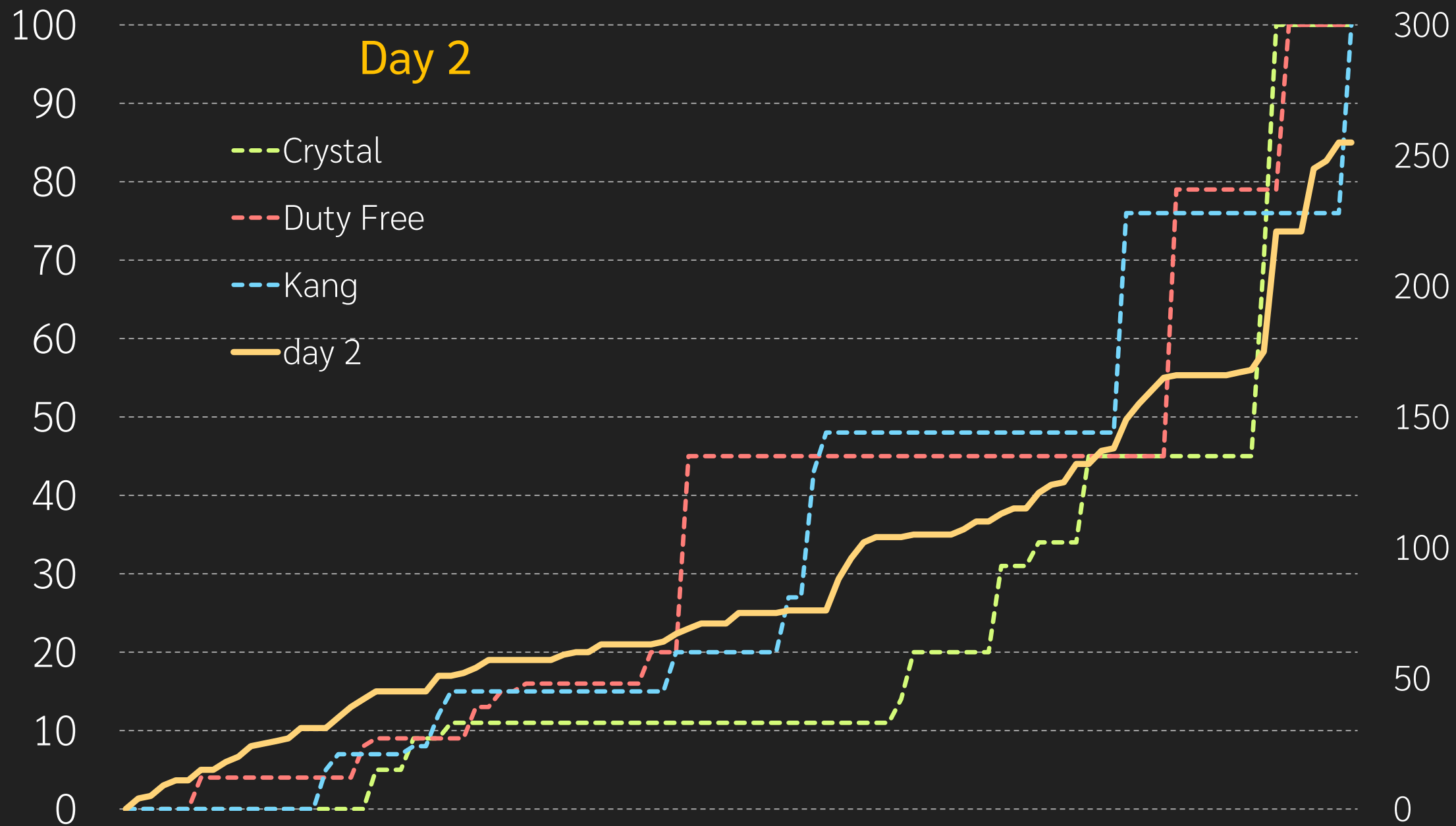
- ใช้ Disjoint Set + (union by size)
 - แต่ละ set คือหมายเลขช่องที่มีกระเป๋าที่อยู่ติดกัน
 - แต่ละ set ให้เก็บหมายเลขช่อง “น้อยสุด” ไว้ด้วย
- พิจารณาคนที่ i จาก 1 ถึง N
 - ดูว่า $S[B[i]]$ วางหรือเปล่า? ถ้าว่าง ก็วางเลย
 - วางแล้ว mark ช่อง $S[B[i]]$ ว่าไม่ว่าง
 - ถ้าช่องซ้ายหรือขวา ($B[i]-1, B[i]+1$) เป็นช่องไม่ว่างด้วย ให้ union set ของ $B[i]$ กับ $B[i]-1$ หรือ $B[i]+1$ ที่ไม่ว่างด้วย ทำให้แต่ละ set คือหมายเลขช่องที่มีกระเป๋าที่อยู่ติดกัน
 - แต่ถ้าไม่ว่าง
 - ถาม set ที่มี $B[i]$ ว่าหมายเลขช่องที่น้อยที่สุดคือช่องอะไร สมมติให้เป็น m
 - เราก็วางกระเป๋าที่ $m-1$ ซึ่งต้องวางแน่นอน (อย่าลืม union set $m-1$ กับ $B[i]$ ด้วย และ อาจจะต้อง union $m-2$ ด้วยถ้าไม่ว่างเช่นกัน)
 - ถ้า m เป็น 1 แสดงว่าไม่มีช่องว่างตั้งแต่ 1 ถึง $B[i]$ เลย ให้ทำการ rearrange



ต่อ 1 คนต้อง 1 find + 2 union เป็นอย่างมาก $O(n \lg n)$

Duty Free Subtask

Subtask	Detail
Subtask 1 $O(N!)$	• ลองวางทุกแบบ แล้ว Simulate การเข้าร้าน
Subtask 2	• $B[i]$ เท่ากันหมด พยายามบอกว่าวาง “ไกลสุด” ก็พอ • คำตอบคือ $N / B[i]$
Subtask 3	• $B[i]$ เรียงจากน้อยไปมาก (ทำให้ไม่ต้องใช้ disjoint set แต่เก็บ max / min ของแต่ละช่วงที่ติดกันก็พอ)
Subtask 4	• $B[i]$ เรียงจากมากไปน้อย เราก็วางกระเป๋าตามนั้นได้เลย
Subtask 5	• ต้องทำเหมือน solution จริง (คิด greedy ออก) แต่ไม่ต้องเขียน disjoint set ก็ได้
Subtask 6	• ต้องทำเหมือน solution จริง (คิด greedy ออก) แต่ time factor เยอะ



Kang Key Idea

- สำหรับแต่ละกรณี K ใด ๆ (ซึ่งเรามีไฟในมือ $N+K$) และต้องทิ้งไฟ K ครั้ง
 - ให้ $C[x]$ คือ จำนวนไฟหมายเลข x ที่มีอยู่ใน มือ ณ เวลาใด ๆ
 - สำหรับการทิ้งไฟแต่ละครั้งนั้น วิธีที่ดีที่สุด คือ เลือก x ที่ $C[x] * x$ มีค่ามากที่สุด แล้วทิ้งไฟ x ทุกใบ
 - แล้วปรับ $C[x]$ ให้เป็น 0 แล้วทำเช่นเดิมซ้ำไปจนครบ K ครั้ง
- Naive Solution
 - ทำ K กรณีแยกกัน แต่ละกรณีก็ต้อง simulate การทิ้งไฟ K ครั้ง แต่ละครั้งก็ต้องเลือก $C[x]*x$ ให้เจอ
 - สำหรับกรณีที่ a เสียแรง $O(M * (\text{เวลาในการเลือก } C[x]*x + \text{เวลาในการปรับปรุงค่า } C[]))$
- Official Solution
 - ทำ K กรณีไปพร้อม ๆ กัน ในกรณีที่ i ต้อง simulate การทิ้งไฟให้ได้เร็ว ๆ
- เป็นข้อที่ต้องใช้ Data Structure คล่อง ๆ รู้ว่าตัวไหนเร็ว ช้า

Kang Official Solution

- ทำงาน M รอบ สำหรับรอบที่ K จะคิดคำตอบของกรณี K ไปเลย โดยเริ่มต้นให้คิดผลรวมของไฟในมือตั้งต้นไว้ก่อนคือ Sum
- แต่ละรอบ จะปรับปรุงค่า C ไปเรื่อย ๆ (ให้สังเกตว่าค่า $C[i]$ ไດ ๆ จะเพิ่มขึ้นเรื่อย ๆ) และค่า C ก็จะเป็นค่า C เริ่มต้นของกรณีที่ K นั้น
- แทนที่จะทำการ simulate ทิ้งไฟ K ใบ จริง ให้มี data structure เพิ่มสองตัว คือ Priority Queue เอาไว้หา $C[x]$ * x และ Set เอาไว้จำว่าเคยทิ้ง X ไปแล้วหรือยัง
 - ตอนจั่วไฟแต่ละใบ ถ้าใบนั้นเคยโดนทิ้งไปแล้ว แสดงว่าจริง ๆ แล้วต้องมาทิ้งตอนนี้จะดีกว่า ให้ไม่ต้องเพิ่ม sum ด้วยไฟใบที่ทิ้งไป แต่ถ้าไม่เคยโดนทิ้ง ก็ให้เพิ่ม sum เข้าไป
 - ใช้ PQ เพื่อเลือก x ที่ $C[x] * x$ มากสุด
 - ถ้า x ที่เลือก ไม่เคยโดนทิ้ง แสดงว่ารอบนี้ต้องทิ้ง X จริง ๆ ก็ทิ้ง X ปรับปรุง PQ (แต่ไม่ต้องปรับปรุง C แล้วให้จำว่าเคยทิ้ง X ไปแล้วแทน) คำนวณผลลัพธ์ของการทิ้ง $C[x]*x$ คือ $sum - C[x]*x$
 - ถ้า x ที่เลือกเคยทิ้งไปแล้ว แสดงว่าในรอบที่มีไฟน้อยกว่านี้เราจะทิ้ง X ก็ยังไม่ต้องทิ้ง ให้ข้ามไปก่อน วนหาจนเจอตัวที่ยังไม่เคยทิ้ง

Kang Example

- ไฟเริ่มต้น = [6, 3, 9, 6, 3] ไฟบนโต๊ะ [4, 6]

กรณีตั้งต้น	กรณี K = 1	กรณี K = 2
Sum = 6+3+9+6+3	Sum = 6+3+9+6+3+4 ทิ้ง 6 ให้ลบ 12 ออกไป	Sum = 6+3+9+6+3+4-12 ทิ้ง 9 ให้ลบ 9 ออกไปด้วย
C[3] = 2 C[6] = 2 C[9] = 1	C[3] = 2 C[4] = 1 C[6] = 2 C[9] = 1	C[3] = 2 C[4] = 1 C[6] = 3 C[9] = 1
ทิ้ง 3 ได้ 6 ทิ้ง 6 ได้ 12 ทิ้ง 9 ได้ 9	ทิ้ง 3 ได้ 6 ทิ้ง 4 ได้ 4 ทิ้ง 6 ได้ 12 ← เลือกทิ้งอันนี้ ทิ้ง 9 ได้ 9	ทิ้ง 3 ได้ 6 ทิ้ง 4 ได้ 4 ทิ้ง 6 ได้ 18 ← เลือกทิ้งอันนี้ ทิ้ง 9 ได้ 9

6 เคยโดนทิ้งไปแล้ว ก็ไม่ต้องบวก 6 เข้ากับ sum

แต่ 6 เคยทิ้งแล้ว ก็ข้ามไป ไปทิ้งตัวมากที่สุดถัดไปแทน คือทิ้ง 9

Kang Subtask

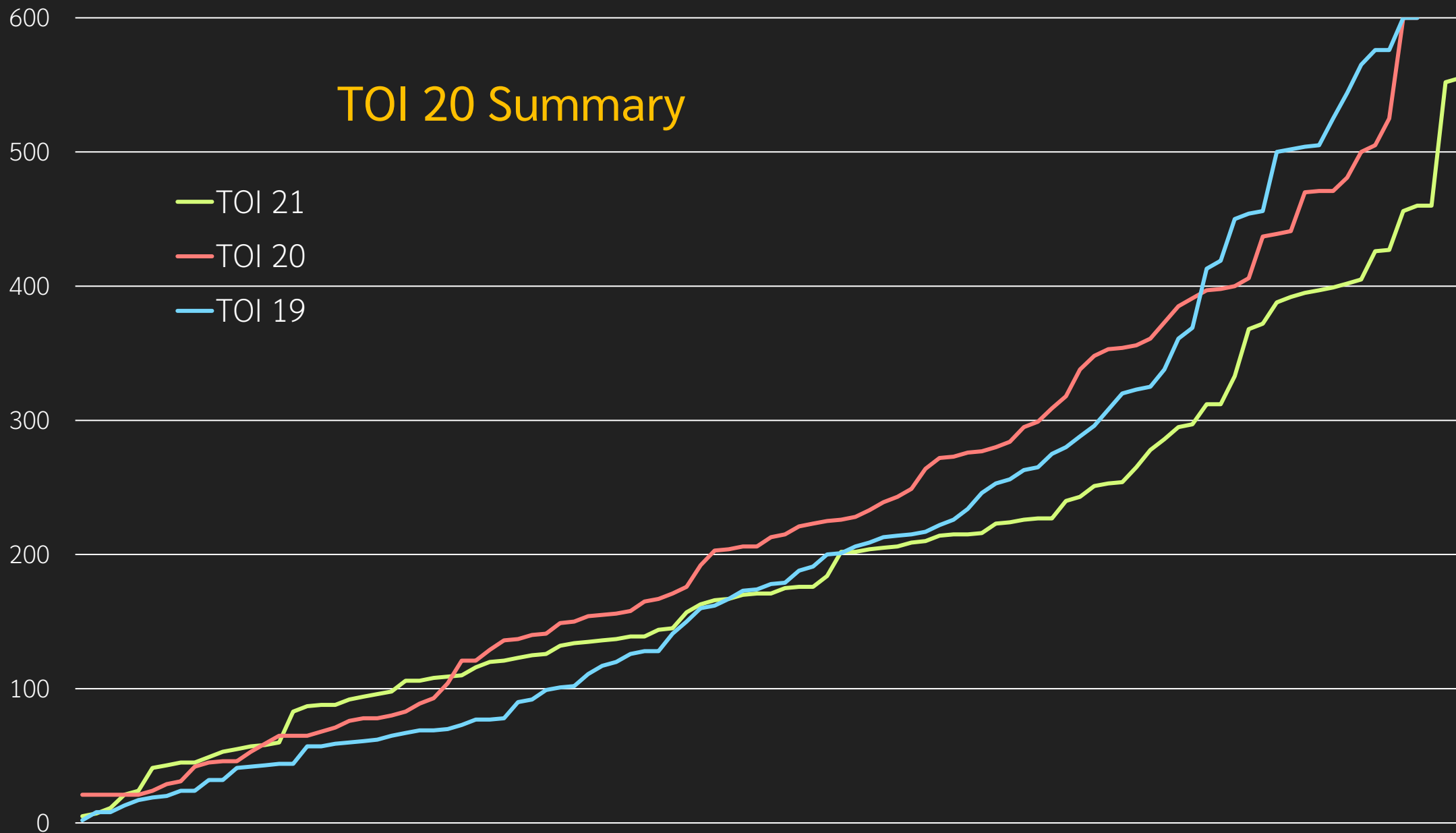
- ต้องเลือก Data Structure ให้ถูก
- Subtask แบ่งตามความเร็วของ data structure ที่เลือก
 - multiset จะช้า
 - ใช้ priority queue แบบไม่ต้องลบข้อมูลมาจำ $\{C[x]*x, x\}$ และใช้ set จำว่าถึง x ไปหรือยัง
- ใช้ set จะไม่ทัน ให้ทำ compression เพื่อใช้ lower_bound บน vector แทน set

Score Summary – Day 2

โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Crystal	7 00:48 min	79	2,221	641
Duty Free	6 00:49 min	93	3,474	1189
Kang	1 04:41 min	83	3,242	822

มีคนไม่ได้คะแนน 1 คน

TOI 20 Summary



Medal Allocation

Medal Allocation

- S5.10 ก่อนพิธีมอบรางวัล TOI นั้น SC ต้องยื่นเรื่องต่อ GA เกี่ยวกับการมอบเหรียญรางวัลให้ผู้เข้าแข่งขันตามคะแนนที่ผู้เข้าแข่งขันได้รับ GA ต้องยืนยันคะแนนและรางวัลของผู้เข้าแข่งขัน ผู้เข้าแข่งขันไม่เกินครึ่งหนึ่งจะได้รับเหรียญรางวัลตามหลักดังนี้
 - ผู้เข้าแข่งขันประมาณหนึ่งในสิบสองของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทอง
 - ผู้เข้าแข่งขันประมาณหนึ่งในหกของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญเงิน
 - ผู้เข้าแข่งขันประมาณหนึ่งในสี่ของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทองแดง
- GA ต้องยืนยันข้อเสนอในการจัดสรรเหรียญรางวัลก่อนให้คณะผู้แทนทั้งหมดรับทราบผลคะแนน

Competitor

- รายชื่อทั้งหมดมีนักเรียน 99 คน (มาเข้าสอบครบทุกคน)
 - Fossil 7 คน
- จำนวนคนที่ใช้คิดในการตัดแบ่งเหรียญรางวัล $= 99 - 7 = 92$ คน

Medal Allocation Proposal (IOI-based)

- การคำนวณเหรียญจะใช้จำนวนคน 92 คน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย $92/12 = 7.6 \sim 8$ คนได้เหรียญทอง
- เกณฑ์คะแนนขั้นต่ำของเหรียญเงินคือคะแนนสูงสุดที่ทำให้มีประมาณ $92/6 = 15.3 \sim 15$ คนได้เหรียญเงิน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองแดงคือคะแนนต่ำสุดที่ทำให้ไม่เกิน $92/2 = 46$ คนได้เหรียญใดเหรียญหนึ่ง แปลว่าต้องมี ทองแดง 23 คน (มากกว่าปีที่แล้ว 1 คน)
 - เนื่องจาก $8+15+23 = 46$
 - เหรียญทองสุดท้าย – เหรียญเงินแรก = 9 คะแนน
 - เหรียญเงินสุดท้าย – เหรียญทองแดงแรก = 27 คะแนน
 - เหรียญทองสุดท้าย – เหรียญทองแดงแรก = 122 คะแนน

Medal Allocation Proposal

(46 คน)	จำนวน	คะแนนต่างจาก rank ถัดไป
ทอง	8	9
เงิน	15	27
ทองแดง	23	1

Issue and Improvement

From TOI20

- มีโจทย์ Interactive

DONE!

การเปลี่ยนแปลงแนวทางการสอบ

- สี่ปีที่ผ่านมา Focus ด้าน Infrastructure ของการสอบ และ รูปแบบข้อสอบ
- ในอนาคต จะ Focus ด้านเนื้อหามากขึ้น

TOI 18 (CMU)	TOI 19 (NU)	TOI 20 (SU)	TOI 21 (KMITL)
• เปลี่ยน Grader มาเป็น CMS • เปลี่ยนมาใช้ Polygon ในการเตรียม Testcases	• Recheck คะแนนจากการรันซ้ำครั้งเดียว • ไม่มีการเปลี่ยนแปลงจากการรันซ้ำ	• เพิ่มเวลาการสอบเป็น 5 ชม. • มีข้อที่มีการให้คะแนนแบบ partial scoring • มีข้อที่มีการให้คะแนนแบบ formula scoring • ข้อ routes • มีการใช้ข้อสอบแบบส่งเฉพาะฟังก์ชัน	• มีโจทย์แบบ interactive • ยกเลิกการรันซ้ำ

Proposal for TOI 22

- เพิ่มขอบเขตเนื้อหา

แนวคิดการปรับเปลี่ยนเนื้อหาอบรม

- ☐ ลดเนื้อหาบางส่วน ปรับบางส่วนเป็นออนไลน์
- ☐ ปัจจุบัน นักเรียนมีพื้นฐานการเขียนโปรแกรมที่ดีขึ้น เนื่องจาก
 - มีวิชาวิทยาการคำนวณ
 - จะมีการเตรียมระบบตรวจโปรแกรมอัตโนมัติเพื่อพัฒนาพื้นฐานของนักเรียนก่อนเข้าค่าย 1
- ☐ ปรับเนื้อหาค่ายดังนี้
 - ปรับปรุง สอน. ค่าย 1 ให้เข้มข้นขึ้น โดยย้ายเนื้อหาโครงสร้างข้อมูลพื้นฐานจาก สอน. ค่าย 2 ไปยัง สอน. ค่าย 1
 - เพิ่มความเข้มข้นของเนื้อหาค่าย สอน. ค่าย 2

TOI-Zero

TOI จะได้ออก
เนื้อหายากขึ้นได้

สรุปการประชุมเชิงปฏิบัติการ การปรับปรุงแบบการคัดเลือก การอบรม และพิจารณาหลักสูตรการอบรม
เพื่อคัดเลือกผู้แทนประเทศไทยไปแข่งขันคณิตศาสตร์ วิทยาศาสตร์โอลิมปิกระหว่างประเทศ

ภายใต้การดำเนินการร่วมกันระหว่างมูลนิธิ สอน. สวท. และ สพฐ.

วันที่ 27 – 28 มกราคม 2567 เวลา 08.30 – 16.30 น.

ณ โรงแรมเดอะ เบิร์เคลีย์ ประตูน้ำ กรุงเทพมหานคร

ค่าย 2 สอน.

เพิ่มความเข้มข้นเนื้อหา ตามที่จะสามารถดำเนินการได้ และตามความสำคัญต่อไปนี้

- Dynamic programming: Recurrences and subproblems, Recurrence evaluation, Linear recurrence, Quadratic recurrence, Recurrences with values: Knapsack & Subset sum, Matrix-chain multiplication
- Basic graph algorithms:
 - BFS: แนวคิดเกี่ยวกับ layers, คุณสมบัติ shortest path, bipartite testing, two-coloring
 - DFS: non-tree edge categorization, directed acyclic graphs & topological sorting
- Proof techniques: (ตาม ioi): Direct proofs, proofs by: counterexample, contraposition, contradiction, mathematical induction
- Greedy algorithms: การพิสูจน์ความถูกต้องของอัลกอริทึมแบบ greedy

การปรับรูปแบบค่าย สสวท และ สอวน

[illegible]

สิ่งที่อยากทำเพิ่มเติมใน TOI 22

- พยายามสรุป scope เนื้อหาที่ “สามารถ” ออกได้ให้ชัดเจน
- เนื้อหาที่ควรจะเริ่มมีในข้อสอบ TOI แล้ว คือ Fenwick Tree